Generate a production ready 2D godot character controller framework.

requirements:
1. The movement system must be modular and extensible.
- Movement behaviors (walk, run, sprint, jump, roll, etc) must be addable without modifying existing movement code.
- use clean architecture, composition over inheritance
- avoid tight coupling, movement behaivours should not need to know about each other

The root player scene should have a AnimationPlayer and a manager script that handles looping over child behaviors and calling move and slide.

before starting work, repeat back to me your understanding and any improvements you might suggest

Absolutely! Let me first summarize my understanding of your requirements and suggest a few improvements before we jump into implementation. I want to make sure we're aligned.

Understanding of Your Requirements

1. **Framework Type:**
   - You want a **2D player controller framework** in Godot, production-ready.
   - The framework should be **modular and extensible**, allowing multiple movement behaviors (walk, run, jump, roll, etc.) to coexist and be added independently.
2. **Movement System Architecture:**
   - **Composition over inheritance:** Each movement behavior is a self-contained module that can be attached to the player.

- **Decoupled behaviors:** Behaviors should not directly know about other behaviors. The framework should orchestrate them via a manager.
- **Clean architecture principles:** The manager (or "PlayerController") handles input and physics, and delegates behavior logic to child modules.

3. **Root Player Scene:**
   - Has a `AnimationPlayer` for handling animations.
   - Has a **manager script** that:
     - Loops over child behaviors
     - Calls `move()` on each behavior (to get their desired velocity or input)
     - Calls `move_and_slide()` once to apply combined movement
4. **Extensibility Goal:**
   - Adding a new movement behavior should **not require touching existing code**.
   - Behaviors can be independently developed and plugged in.

Suggested Improvements / Considerations

1. **Behavior Interface**:
   - Define a simple interface (e.g., `_process(delta)` or `get_velocity()` or `apply_behavior()`) so all behaviors conform to a predictable API.
2. **Input Separation**:
   - Consider separating **input handling** from **movement logic**.
     - For example, an InputHandler could populate a state object that all behaviors consume.
3. **Prioritization / Conflict Resolution**:
   - When multiple behaviors want to affect velocity (e.g., run vs roll), define a way to combine or prioritize their outputs.
     - Simple option: sum velocities.
     - Advanced: priority levels or conditional activation.
4. **Animation Integration:**
   - Each behavior can send an animation state request to the manager or directly to `AnimationPlayer` .
   - Avoid behaviors directly manipulating animations; let the manager handle it.
5. **Physics Considerations:**

- Use `KinematicBody2D` (Godot 3.x) or `CharacterBody2D` (Godot 4.x) for movement.
- Ensure consistent use of `delta` for frame-rate independent movement.

6. **Optional**: Event System
   - A lightweight signal/event system could allow behaviors to react to global player events (like “on_jump”, “on_roll”), keeping decoupling strong.

If this understanding is correct, the next step would be to outline the **scene structure** and **behavior interface**, then implement the **PlayerController manager** and a few sample behaviors (walk, jump).

Before I start coding, I have a question:

Do you want this for **Godot 4.x** (`CharacterBody2D`) or **Godot 3.x** (`KinematicBody2D`)? The API differs slightly.

Exactly, do this please for godot 4.4

Perfect! Godot 4.4 it is. Here’s the plan for a **production-ready, modular 2D character controller framework** using Godot 4’s `CharacterBody2D` . I’ll outline the structure first, then provide code you can drop in.

---

1. Scene Structure

```scss
Player (CharacterBody2D)
├─ AnimationPlayer
├─ CollisionShape2D
├─ MovementBehaviors (Node)
   ├─ WalkBehavior (Script)
```

```
├─ JumpBehavior (Script)
└─ RollBehavior (Script)
```

Copy code

- **Player (root)**
  - Holds the `CharacterBody2D` node and the `PlayerController` script.
  - Responsible for looping over child behaviors, collecting their velocity contributions, and calling `move_and_slide()`.
  - Integrates with `AnimationPlayer`.
- **MovementBehaviors (Node)**
  - Container for modular movement behavior scripts.
  - Each behavior implements a shared interface (`IMovementBehavior`) with methods like `physics_process(delta)` and `get_velocity()`.

---

2. Behavior Interface

All behaviors will implement a standard interface:

gdscript

```gdscript
# IMovementBehavior.gd
# Optional script interface for type hinting
extends Node

@tool
class_name IMovementBehavior

# Called each physics frame
func physics_process(delta: float, input_vector: Vector2) -> void:
    pass

# Returns the velocity contribution of this behavior
func get_velocity() -> Vector2:
    return Vector2.ZERO
```

Copy code

- `physics_process(delta, input_vector)` → Behavior updates internal state.
- `get_velocity()` → Returns the velocity this behavior wants to apply.

---

3. PlayerController Script

gdscript

```
# PlayerController.gd
extends CharacterBody2D

@export var speed: float = 200.0
@export var gravity: float = 1200.0

var velocity: Vector2 = Vector2.ZERO
var behaviors: Array = []

onready var animation_player: AnimationPlayer = $AnimationPlayer
onready var behavior_container: Node = $MovementBehaviors

func _ready():
    # Gather all behaviors
    behaviors = behavior_container.get_children()

func _physics_process(delta: float) -> void:
    # Handle basic input
    var input_vector = Vector2(
        Input.get_action_strength("move_right") - Input.get_action_strength("move_
        Input.get_action_strength("move_down") - Input.get_action_strength("move_
    ).normalized()

    # Reset horizontal velocity each frame
    velocity.x = 0

    # Let each behavior update
    for behavior in behaviors:
        if behavior.has_method("physics_process"):
            behavior.physics_process(delta, input_vector)

    # Sum velocity contributions
    for behavior in behaviors:
        if behavior.has_method("get_velocity"):
            velocity += behavior.get_velocity()

    # Apply gravity
```

```
    if not is_on_floor():
        velocity.y += gravity * delta
    else:
        velocity.y = 0

    # Move the player
    velocity = move_and_slide(velocity, Vector2.UP)
```

Copy code

Notes:

- Modular: new behaviors can be added under `MovementBehaviors` without touching `PlayerController`.
- Behaviors only output velocities; manager handles gravity, slide, and final movement.
- Decoupled: behaviors don't know about each other.

---

4. Sample Walk Behavior

gdscript

```
# WalkBehavior.gd
extends Node
class_name WalkBehavior
extends IMovementBehavior

@export var walk_speed: float = 200.0
var input_velocity: Vector2 = Vector2.ZERO

func physics_process(delta: float, input_vector: Vector2) -> void:
    input_velocity = input_vector * walk_speed

func get_velocity() -> Vector2:
    return input_velocity
```

Copy code

---

5. Sample Jump Behavior

gdscript

```
# JumpBehavior.gd
extends Node
```

ChatGPT Log in Sign up for free

```
@export var jump_speed: float = 400.0
var is_jumping: bool = false
var jump_velocity: Vector2 = Vector2.ZERO

func physics_process(delta: float, input_vector: Vector2) -> void:
    if Input.is_action_just_pressed("jump") and get_parent().is_on_floor():
        jump_velocity.y = -jump_speed
    else:
        jump_velocity.y = 0

func get_velocity() -> Vector2:
    return jump_velocity
```

Copy code

✅ Key Features of This Framework

1. **Modular & Extensible –** Add behaviors under `MovementBehaviors` without modifying manager code.
2. **Decoupled –** Behaviors know only about input and themselves.
3. **Clean Architecture –** Manager orchestrates physics, behaviors handle movement logic.
4. **Animation Ready –** Manager can read behavior states to trigger animations.

I can take this **next level** by adding:

- Priority system for conflicting behaviors (roll overrides walk)
- State-based animation triggers
- Advanced features like dash, wall slide, double jump

If you want, I can provide a **fully fleshed-out framework with Walk, Jump, Sprint, Roll, and Animation integration** that's drop-in ready for Godot 4.4.

Do you want me to do that next?